

LLM Alignment:
How Language Models Learn
from Human Preferences
From Next-Token Prediction to Preference Optimisation

Suryanarayana Sankagiri
Wadhvani School of Data Science and AI
Indian Institute of Technology Madras

[Personal webpage](#) | [Email](#)

DA5453 — Learning from Human Preferences
First offered at IIT Madras, July–November 2026

[Course webpage](#)

September 2026

AI-use disclaimer. This document was generated with substantial assistance from AI tools and may contain hallucinations, mathematical errors, omissions, or passages that are difficult to read. Nevertheless, I, Suryanarayana Sankagiri, accept full responsibility for the material presented here. Readers are encouraged to contact me about any suspected error or unclear passage.

About these notes

These notes cover the LLM alignment module of DA5453, *Learning from Human Preferences*. They build directly on Module 1: the same probabilistic models that turn noisy comparisons into estimates of latent utility now provide training signals for language models. Here, alignment concerns adapting a model’s responses to the behaviour represented by demonstrations and preference judgements. LLMs are treated largely as black boxes. We introduce the probabilistic input–output structure needed to formulate the learning objectives and explain how they are evaluated, leaving detailed architectures and training systems outside the scope of the notes.

The narrative begins with next-token prediction and supervised fine-tuning, then introduces preference data and Bradley–Terry reward models. It next develops KL-regularised policy optimisation using a learned reward, before showing how the same regularised formulation leads to direct preference optimisation, which trains a language model directly from preferred and dispreferred responses. Each chapter closes with a short summary and its principal equations, and historical remarks are collected at the end.

Contents

Notation	iv
1 Language models, pretraining, and instruction following	1
1.1 Language models in one page	1
1.2 Notation and the policy view	2
1.3 Pretraining by maximum likelihood	3
1.4 Instruction following and supervised fine-tuning	5
1.5 Why supervised fine-tuning is not enough	6
1.6 Summary and key equations	8
2 Learning a reward model from comparisons	9
2.1 From observed comparisons to a scoring rule	9
2.2 Why the Bradley–Terry model must be contextual	9
2.3 Constructing a neural reward model	10
2.4 From comparison probabilities to a likelihood	11
2.5 What the loss learns	12
2.6 How comparison data are constructed	12
2.7 Preference data in practice	13
2.8 What does the learned score represent?	14
2.9 Evaluating generalisation and shortcuts	15
2.10 Summary and key equations	16
3 Improving a policy with a learned reward	17
3.1 From response scores to policy value	17
3.2 What pure reward maximisation asks for	17
3.3 Why a policy gradient is needed	18
3.4 How a response score reaches token probabilities	19
3.5 Why pure reward optimisation needs a reference	20
3.6 Kullback–Leibler divergence for response distributions	21
3.7 The KL-regularised objective	22
3.8 Summary and key equations	23
4 From the optimal policy to direct preference optimisation	24
4.1 A warm-up: entropy-regularised probability allocation	24
4.2 Solving the KL-regularised reward objective	25
4.3 From an optimal policy to a preference likelihood	26
4.4 What the DPO margin asks the policy to change	27
4.5 Computing a DPO training step	28
4.6 What DPO changes, and what it leaves unchanged	28
4.7 Summary and key equations	29
Historical remarks	30
References	33

Notation

Symbol	Meaning
$z = (z_1, \dots, z_T), z_{<t}$	Token sequence and the tokens preceding position t , used for pretraining.
$x, y = (y_1, \dots, y_T)$	Prompt and response; complete responses include a terminal token.
$y_{<t}, V$	Response tokens preceding position t , and token vocabulary.
a_v	Logit assigned to token v before softmax normalisation.
π_θ	Language-model policy with trainable parameters θ .
q	Fixed reference policy, usually obtained after supervised fine-tuning.
$r_\phi(x, y)$	Reward model with parameters ϕ ; abbreviated to r when fixed.
$\mathcal{D}_{\text{SFT}}, \mathcal{D}_{\text{pref}}$	Demonstration dataset and preference dataset.
μ	Distribution of prompts used in the policy objective.
y^+, y^-	Responses preferred and dispreferred in a recorded comparison.
$\sigma(u) = (1 + e^{-u})^{-1}$	Logistic function.
$\beta > 0$	Strength of the KL penalty; the same convention is used throughout.
$\log, \mathbb{E}$	Natural logarithm and expectation.

Language models, pretraining, and instruction following

Before studying how human preferences can change a language model, we need a working description of the model itself. We begin with next-token prediction, then use the probabilities it supplies to formulate pretraining and supervised fine-tuning. These objectives provide the starting point for learning from comparisons.

1.1 Language models in one page

A *language model* is a generative model of language: it assigns probabilities to possible pieces of text and can generate new text from those probabilities. Language models power general-purpose AI chatbots such as ChatGPT and Claude. Modern systems built around them can also search, call tools, write and execute code, and carry out multi-step tasks; such systems are often described as agents. Those additional capabilities matter in practice, but they are not needed for the mathematical development here. We will treat the language model itself as a black box: text goes in, and the model returns a probability distribution over what text could come next.

Many kinds of language model have been studied. We focus on autoregressive models built with the *Transformer* architecture (Vaswani et al. 2017). At a high level, the computation has four stages. A tokenizer first divides text into tokens, which may be words, parts of words, punctuation, or special markers. Each token is then represented by a learned numerical vector called an embedding.

These vectors pass through a stack of Transformer blocks. Within each block, attention allows the representation at a position to incorporate information from relevant earlier positions, while other neural-network operations further transform it. By the end of the stack, the vector at the current position contains information gathered from the text that came before it. This final representation is converted into one score for every possible next token, and those scores are normalised into probabilities. For our purposes, the internal layers may remain hidden; what matters is the distribution they compute.

The fundamental task is *next-token prediction*. Given the text so far, the model predicts a distribution over the next token. To generate text, it selects a token from this distribution, appends that token to the text, recomputes the distribution, and repeats. Each selection changes the context for the next one. A model that generates in this way is called *autoregressive*: the next output depends on the outputs already generated. Although each individual prediction concerns only one token, repeating the operation produces a complete sentence, paragraph, or conversation. A special terminal token tells the model when to stop.

1.2 Notation and the policy view

We first describe a generic token sequence, without yet separating a prompt from a response. Let V be the model’s finite vocabulary and let

$$z = (z_1, \dots, z_T)$$

be a token sequence ending at its first terminal token $\text{EOS} \in V$. The prefix before position t is $z_{<t} = (z_1, \dots, z_{t-1})$; at $t = 1$, this is the empty prefix, represented in practice by a start marker. Given this prefix, a model with parameters θ assigns a probability to every possible next token:

$$\pi_\theta(v \mid z_{<t}) \geq 0, \quad \sum_{v \in V} \pi_\theta(v \mid z_{<t}) = 1. \quad (1.1)$$

The parameters θ are the numerical weights learned during training. The vertical bar means “given”, so $\pi_\theta(v \mid z_{<t})$ is the probability that the next token is v , given the preceding tokens. This is a complete probability distribution over the vocabulary, not merely a ranking of plausible tokens.

The notation π is deliberate. In a sequential decision problem, a *policy* specifies a distribution over actions in each state. Here the state is the current token prefix and the action is the next token. Appending the selected token produces the next state, and selecting EOS terminates the sequence. A generative language model is therefore a stochastic policy over tokens. Later chapters change this policy using rewards and preferences without requiring a different notion of generation.

1.2.1 From network scores to an exact distribution

For a fixed prefix, the network produces one real-valued score a_v , or *logit*, for every $v \in V$. The softmax function converts these scores into probabilities:

$$\pi_\theta(v \mid z_{<t}) = \frac{\exp(a_v)}{\sum_{u \in V} \exp(a_u)}. \quad (1.2)$$

Exponentiation makes every weight positive. The denominator is the normalisation constant: it is computed by summing the exponentiated score of every token in the vocabulary. Consequently, once the prefix and parameters are fixed, the model’s next-token distribution is known explicitly, including its normalising constant. Subtracting the largest score from all scores before exponentiation improves numerical stability and leaves the probabilities unchanged.

The ratio of two probabilities is $\exp(a_v - a_u)$, so only score differences matter. Adding the same constant to every logit leaves the distribution unchanged.

1.2.2 Sampling, randomness, and temperature

Knowing the next-token distribution is not yet enough to determine which token will be produced. A *decoding rule* specifies how the model turns that distribution into a choice. Under ordinary sampling, we may think of $[0, 1)$ as being divided into intervals whose lengths are the token probabilities. We draw a uniform random number U and select the token whose interval contains it. After that token is appended, the model computes a new distribution and makes a new draw.

For example, if three tokens have probabilities 0.6, 0.3, and 0.1, the intervals may be $[0, 0.6)$, $[0.6, 0.9)$, and $[0.9, 1)$; a draw $U = 0.72$ selects the second token.

The decoding rule may first modify the model probabilities. A positive *temperature* τ , for example, gives

$$p_\tau(v \mid z_{<t}) = \frac{\exp(a_v/\tau)}{\sum_{u \in V} \exp(a_u/\tau)}.$$

Lower temperatures concentrate probability on the highest-scoring tokens; higher temperatures make the distribution more even. At $\tau = 1$, this is the original model distribution.

Top-k sampling retains the k most probable tokens. *Nucleus sampling* retains the smallest set of highest-probability tokens whose total probability reaches a chosen threshold. Both rules renormalise the retained probabilities before sampling. Greedy decoding simply chooses a token with the largest score. The model distribution, the decoding rule, and the random draw are therefore distinct parts of generation.

Remark 1.2.1 (Where does the random number come from?). Software normally uses a pseudorandom-number generator. When a model is run locally, for example through a Hugging Face library, the generator begins from a seed. The seed may be chosen by the user for reproducibility or obtained from the operating system, which gathers physical and timing-based entropy. The generator then produces a deterministic stream of numbers that behaves like independent uniform draws. Hosted models use the same basic mechanism, although the provider may not expose the seed or the exact sampling implementation. The language model supplies the categorical probabilities, and the pseudorandom-number generator supplies the draw used to select a token. Given the same algorithm and seed, the generator reproduces the same pseudorandom stream.

1.2.3 From next-token probabilities to sequence probabilities

The next-token distributions also determine the probability of a complete sequence. Applying the chain rule gives

$$\pi_{\theta}(z) = \prod_{t=1}^T \pi_{\theta}(z_t \mid z_{<t}), \quad \log \pi_{\theta}(z) = \sum_{t=1}^T \log \pi_{\theta}(z_t \mid z_{<t}). \quad (1.3)$$

The product does not assume that tokens are independent. Each factor uses the entire available prefix. Including EOS means that a sequence probability includes the decision to stop. We assume that generation terminates with probability one. A fixed length limit can also be incorporated by forcing EOS with probability one at the final permitted position; in that case, the product uses this modified stopping rule.

1.3 Pretraining by maximum likelihood

We now turn from generation to learning. Pretraining uses ordinary text rather than prompt–response pairs. Let the corpus contain token sequences $z^{(1)}, \dots, z^{(M)}$, where sequence i has length T_i . Treating these sequences as independent observations under the model, the likelihood assigned to the observed corpus is

$$L_{\text{pre}}(\theta) = \prod_{i=1}^M \prod_{t=1}^{T_i} \pi_{\theta}(z_t^{(i)} \mid z_{<t}^{(i)}).$$

Recall the maximum-likelihood principle: choose parameters that make the observed data as probable as possible under the model. Products of many small probabilities are inconvenient, so we take logarithms. If $N = \sum_i T_i$ is the number of predicted tokens, maximising the corpus likelihood is equivalent to minimising the average negative log likelihood

$$\mathcal{L}_{\text{pre}}(\theta) = -\frac{1}{N} \sum_{i=1}^M \sum_{t=1}^{T_i} \log \pi_{\theta}(z_t^{(i)} \mid z_{<t}^{(i)}). \quad (1.4)$$

For every position in a recorded text, the preceding tokens form the input and the token at that position forms the target. The loss favours higher conditional probability for the recorded targets. The targets are obtained directly from the text, without requiring a person to annotate a separate answer for every prefix. A corpus containing N tokens consequently supplies approximately N next-token prediction cases.

Because the prediction targets come from the corpus itself, pretraining is often described as *self-supervised learning*. Collecting, filtering, and tokenising the corpus remain separate parts of constructing the training data.

1.3.1 How one recorded sequence supplies many training cases

Consider the recorded sequence `Rain falls .` followed by EOS. One training case uses `Rain` as the visible prefix and `falls` as the target. A second uses `Rain falls` as the prefix and the period as the target. A third uses the prefix ending in the period and EOS as the target. The same document therefore supplies a prediction target at every position.

These cases need not be processed one after another during training. The complete recorded sequence is already known. A Transformer can compute the representations and next-token scores for many positions in parallel. The output at one position predicts the following token, while a causal attention mask prevents that position from looking to its right. Thus the target at position t is the observed z_t , but the corresponding prediction may use only $z_{<t}$. During generation the future is not recorded, so the model must sample one token, append it, and run the next step. Training can be parallel across positions even though generation is sequential.

1.3.2 The loss and its gradient

At one position, let a_v be the logit for token v , let

$$p_v = \frac{e^{a_v}}{\sum_{u \in V} e^{a_u}},$$

and let w be the token that actually occurred. The loss for this position is

$$\ell(a, w) = -\log p_w = -a_w + \log \sum_{u \in V} e^{a_u}.$$

This is also the *cross-entropy loss*: for a target distribution t and a predicted distribution p , cross-entropy is $-\sum_v t_v \log p_v$. Here the target places probability one on the observed token w , so the expression reduces to $-\log p_w$.

Its derivative with respect to each logit is

$$\frac{\partial \ell}{\partial a_v} = p_v - \mathbf{1}\{v = w\}.$$

The indicator is one only for the observed token. A gradient step in these logits raises the observed token's logit and lowers the others: for $v = w$, the derivative is $p_w - 1 \leq 0$; for every other token it is $p_v \geq 0$. In a language model, the logits depend on shared parameters. Backpropagation combines these logit derivatives across positions and carries them through the Transformer and its token embeddings to θ . On a batch of text, gradient descent makes the update

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{batch}}(\theta),$$

where $\eta > 0$ is the learning rate. The notation changes scale from one position, to a batch, to the full corpus, but the learning signal remains the same: increase the likelihood of the recorded next tokens.

1.3.3 Why the same objective can be used at large scale

At first sight, learning a useful distribution over all possible text seems intractable. The number of possible token sequences grows exponentially with their length, and an effective prediction may depend on information appearing much earlier in the text. The training procedure becomes manageable because it never stores a separate parameter for every complete sequence. The same Transformer parameters compute the next-token distribution for every prefix, so an update obtained from one occurrence can affect predictions in many other contexts. The target for each prediction is read from the corpus itself, and the calculations for different documents and different positions within a document can be divided across many processors.

The resulting computation is nevertheless very large. Increasing the amount and variety of text supplies more observed contexts; increasing the number of model parameters enlarges the class of conditional distributions that can be fitted; and increasing the computational budget permits more of these observations to be used in optimisation. GPT-3, for example, had 175 billion parameters and was trained on 300 billion sampled tokens (Brown et al. 2020). Across those token positions, the loss was always of the form shown in Equation (1.4). Repeated optimisation of this loss can fit grammatical patterns, factual associations, styles of writing, common procedures, and dependencies that extend across long passages.

1.3.4 The base model produced by pretraining

This stage is called *pretraining* because it takes place before the model is adapted to a particular task or mode of interaction. The resulting model is called a *base model*. Pretraining usually accounts for most of the computation used to train such a model; the term describes the order of the training stages, not the amount of training completed.

A base model can generate fluent passages and can often answer questions, translate, summarise, or imitate examples supplied in its context. GPT-3 demonstrated many such tasks under zero-shot and few-shot prompting, without a separate parameter update for each task (Brown et al. 2020). These behaviours can be learned because the pretraining corpus contains questions and answers, translations, code, conversations, worked examples, and other recurring forms of text.

In these notes, an *assistant* is a language model used through an interface in which one piece of text is presented as a user’s request and the generated continuation is presented as the response. For example, the request may ask for a paragraph to be summarised, for a sentence to be translated, or for a calculation to be explained. In each case, the response is judged by whether it fulfils the request, not only by whether it resembles text that could follow the request in the pretraining corpus.

The pretraining data do not mark every passage by whether it would be a suitable response in such an interaction. A document may contain a question followed by an answer, an unanswered question, several conflicting answers, or a discussion in which the question is quoted for another purpose. Next-token likelihood learns the frequencies and regularities of all these forms. It therefore provides no direct observation of whether a generated continuation has preserved every constraint in a particular user’s request.

Brown et al. provide an example. When asked to correct the sentence “Mary arranged for, in St. Louis, John to rent a house cheap”, GPT-3 returned “Mary arranged for John to rent a house in St. Louis.” The response repairs the word order but omits that the house should be cheap (Brown et al. 2020). This error illustrates the distinction: the output is a grammatical continuation in the form of a correction, but it does not preserve all the information in the input.

Training performed after pretraining is generally called *post-training*. The first post-training method considered here is supervised fine-tuning on examples of requests and suitable responses.

1.4 Instruction following and supervised fine-tuning

The notation z was useful while discussing ordinary text. For an assistant, it helps to separate what the user supplies from what the model generates. Let x denote the *prompt*: all the text supplied to the model before it begins the response. Let $y = (y_1, \dots, y_T)$ denote a complete response, including EOS. The model assigns the response the conditional probability

$$\pi_{\theta}(y \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}), \quad \log \pi_{\theta}(y \mid x) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (1.5)$$

This is the same chain rule as in (1.3). The notation now separates the supplied prompt from the response. In the factor for position t , the model receives the complete prompt x and the preceding response tokens $y_{<t}$, and assigns a probability to the recorded next response token y_t . Multiplying these probabilities gives the probability of producing the complete response, including its stopping token.

A prompt can describe a task in ordinary language or demonstrate a pattern. In *few-shot prompting*, x contains several worked examples followed by a new input, and the response y continues the demonstrated pattern. Zero-shot, one-shot, and few-shot refer to zero, one, or several demonstrations in the prompt. GPT-3 showed how strongly this use of context can elicit task behaviour from a base model (Brown et al. 2020). The model parameters remain fixed; only the prompt changes.

Supervised fine-tuning (SFT) starts from a base model and continues training on *demonstrations*: examples of how the model should respond to particular prompts. A demonstration may be written by a person, adapted from an existing task, or generated by another model and retained after filtering. The term *supervised* refers to supplying a selected target response for each prompt. Write the demonstration dataset as

$$\mathcal{D}_{\text{SFT}} = \{(x_i, y_i)\}_{i=1}^n,$$

where x_i is a prompt and $y_i = (y_{i,1}, \dots, y_{i,T_i})$ is the demonstrated response. For a fixed candidate value of θ , the conditional likelihood of the dataset is

$$L_{\text{SFT}}(\theta) = \prod_{i=1}^n \pi_{\theta}(y_i | x_i) = \prod_{i=1}^n \prod_{t=1}^{T_i} \pi_{\theta}(y_{i,t} | x_i, y_{i,<t}). \quad (1.6)$$

The first product combines the probabilities assigned to the demonstrated responses. The second equality expands each response probability by the chain rule. Maximum conditional likelihood chooses the parameters that assign the greatest conditional likelihood to the demonstrated responses, treating the prompts as given. As in pretraining, it is more convenient to minimise a negative log likelihood. If $N_{\text{SFT}} = \sum_i T_i$ is the total number of response tokens, the token-averaged loss is

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N_{\text{SFT}}} \sum_{i=1}^n \log \pi_{\theta}(y_i | x_i) = -\frac{1}{N_{\text{SFT}}} \sum_{i=1}^n \sum_{t=1}^{T_i} \log \pi_{\theta}(y_{i,t} | x_i, y_{i,<t}). \quad (1.7)$$

Just as in pretraining, supervised fine-tuning therefore takes place by optimising a next-token prediction loss. The change lies in the training data and in the positions at which the loss is evaluated. The prompt supplies context, and the demonstrated response supplies the target tokens. When prompts and responses are stored as one token sequence, a loss mask excludes the prompt tokens and any padding used to assemble a batch. The terminal token is included among the response targets so that the model can learn where an answer should end.

During this calculation the demonstrated response is supplied rather than sampled from the model. At response position t , the model sees the prompt and the preceding demonstrated tokens $y_{<t}$; the recorded token y_t is the target. Its negative log probability contributes one term to the loss. Backpropagation changes θ , and hence changes the model probabilities, while the demonstration itself remains fixed.

Pretraining and SFT thus use the same likelihood construction for different datasets. Pretraining fits token sequences drawn from a broad text corpus. SFT raises the probability of responses selected as demonstrations for particular prompts. Depending on the demonstrations, this can teach the model to recognise requests, follow output formats, preserve instructions, decline some requests, or use a specified conversational style.

1.5 Why supervised fine-tuning is not enough

Supervised fine-tuning tells the model which responses to imitate. It is less direct about how to choose among several plausible responses. A demonstration records one selected answer, but it does not explain why that answer was selected instead of another one that the model might produce. Several answers may be acceptable, demonstrations may be imperfect, and some failures become visible only after the fine-tuned model is used. SFT can still produce a capable assistant when the demonstrations cover the desired behaviour well; the issue is that demonstrations leave some useful information unrecorded.

One remedy is to collect additional and better demonstrations. Another is to generate several candidate responses to the same prompt and ask an evaluator to compare them. A comparison can record that

one response preserves a constraint that another misses, or that a correct derivation is less suitable for a beginner than an explanation that defines its symbols. The evaluator can make this judgement without composing a replacement response. A preferred response is only the better of the alternatives shown; it need not be fully correct or satisfactory in absolute terms. Preference data therefore supplement rather than replace demonstrations.

For a simple numerical example, suppose a demonstration dataset for one prompt contains response A eight times and response B twice. If these are the only possible responses and $p = \pi(A \mid x)$, their likelihood is $p^8(1 - p)^2$, which is maximised at $p = 0.8$. Suppose instead that evaluators comparing the two responses prefer B because it preserves an important condition omitted by A . That comparison is information not contained in the demonstration counts. It could be incorporated by replacing or reweighting the demonstrations, or it could be recorded directly as the preference $B \succ A$.

The InstructGPT experiments provide one empirical example. In the reported evaluation, labelers preferred outputs from models trained further with comparison data over outputs from the corresponding SFT models (Ouyang et al. 2022). The conclusion is specific to the prompts, labelers, training procedures, and evaluation criteria in that study; it does not imply that preference training corrects every limitation of supervised fine-tuning.

The three-stage construction used by Ouyang et al. (2022) is summarised in Figure 1.1. First, demonstrations are used for supervised fine-tuning. Second, responses sampled from a model are ranked by evaluators, and a reward model is fitted to these comparisons. Third, the language-model policy is updated using the fitted reward. In the mathematical treatment that follows, the policy update is regularised by comparison with a fixed reference policy.

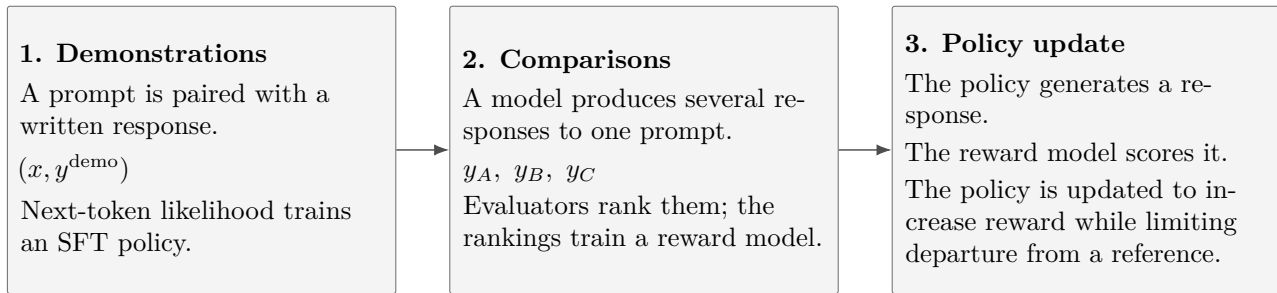


Figure 1.1: The three training stages used in InstructGPT, redrawn from the method described by Ouyang et al. (2022).

The next chapter develops the reward model used in the second stage. Chapter 3 studies the regularised policy objective, and Chapter 4 shows how direct preference optimisation uses the comparison data without fitting a separate reward model.

1.6 Summary and key equations

A language model is a generative model that repeatedly predicts and selects the next token. A Transformer computes the next-token logits, softmax turns them into an explicit model distribution, and the decoding rule determines how the next token is selected. The resulting generator is a policy whose state is the current text and whose action is the next token.

Pretraining applies maximum likelihood to ordinary token sequences, using each recorded token as a target conditional on its prefix. The resulting base model can continue text and can perform many tasks when they are expressed through prompts. Supervised fine-tuning uses the same next-token loss on selected prompt–response demonstrations. Comparisons add observations about the relative quality of responses generated for the same prompt and can be used to fit a reward model or a direct preference objective.

Key equations

$$\begin{aligned}\pi_{\theta}(v \mid z_{<t}) &= \frac{e^{a_v}}{\sum_{u \in V} e^{a_u}}, \\ \pi_{\theta}(z) &= \prod_{t=1}^T \pi_{\theta}(z_t \mid z_{<t}), \\ \mathcal{L}_{\text{pre}}(\theta) &= -\frac{1}{N} \sum_{i,t} \log \pi_{\theta}(z_t^{(i)} \mid z_{<t}^{(i)}), \\ \pi_{\theta}(y \mid x) &= \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}), \\ \mathcal{L}_{\text{SFT}}(\theta) &= -\frac{1}{N_{\text{SFT}}} \sum_i \log \pi_{\theta}(y_i \mid x_i).\end{aligned}$$

Learning a reward model from comparisons

Chapter 1 ended with observations of the form “response A was preferred to response B for this prompt.” A single observation is local: it tells us about two particular responses to one particular prompt. To use many such observations for training, we need a rule that can also evaluate responses which no labeler has seen. This chapter explains how a finite collection of comparisons can be turned into such a rule.

The rule is called a *reward model*. The name comes from its later use as a training signal. It reads prompt–response pairs, while its supervision consists of comparison outcomes rather than numerical rewards supplied by the evaluator. From the viewpoint of preference learning, it is a statistical model for predicting how other responses to a prompt would be compared.

2.1 From observed comparisons to a scoring rule

The first step is to decide how one comparison will be recorded. For a prompt x , suppose an evaluator is shown two responses and selects one. We write the resulting observation as

$$(x, y^+, y^-),$$

where y^+ is the response selected as better and y^- is the other response. The signs record the outcome of this comparison. They do not assert that y^+ is an ideal answer or that y^- has no merit. Both responses may be good, or both may be poor. We consider strict binary choices; ties require a separate treatment and are not encoded by this triple.

Collecting n observations gives the preference dataset

$$\mathcal{D}_{\text{pref}} = \{(x_i, y_i^+, y_i^-)\}_{i=1}^n. \quad (2.1)$$

For example, two summaries may both be fluent, while only one preserves an important qualification in the source. The label records the evaluator’s choice under the supplied instructions and interface.

The central difficulty appears as soon as we look beyond the recorded rows. A language model can generate many responses that are absent from the dataset, including responses to entirely new prompts. We therefore seek a function

$$r_\phi(x, y) \in \mathbb{R}$$

whose parameters ϕ are shared across all observations. The value $r_\phi(x, y)$ is a score for response y in the context of prompt x . If the model has learned useful regularities, comparisons involving some responses can inform its scores for related responses that were not compared during training. This ability to generalise beyond the recorded comparisons is why a model is needed.

2.2 Why the Bradley–Terry model must be contextual

The natural starting point is the Bradley–Terry model from Module 1. For a fixed collection of items, it assigns

$$\mathbb{P}(i \succ j) = \sigma(\theta_i - \theta_j).$$

One could try to use this model directly by treating every prompt–response pair in the preference dataset as a separate item with its own parameter. This construction has two limitations. First, comparisons are normally made only within a prompt, so the comparison graph has no edges connecting different

prompts; even within one prompt it may be disconnected. Second, a newly generated response is a new item for which no parameter has been estimated. A table of item-specific utilities can remember the responses in the table, but it cannot score an arbitrary piece of text.

A reward model replaces the free utility θ_i by a function of the prompt and response. For two candidate responses y_a and y_b to prompt x , the *contextual Bradley–Terry model* is

$$\mathbb{P}_\phi(y_a \succ y_b \mid x) = \frac{e^{r_\phi(x, y_a)}}{e^{r_\phi(x, y_a)} + e^{r_\phi(x, y_b)}} = \sigma(r_\phi(x, y_a) - r_\phi(x, y_b)). \quad (2.2)$$

Equation (2.2) uses the same logistic link as the Bradley–Terry model in Module 1. Its systematic utility is supplied by the shared function $r_\phi(x, y)$, computed from the text of each prompt–response pair.

The dependence on x is necessary because suitability is conditional on the request. The response “Paris” is appropriate for a prompt asking for the capital of France and inappropriate for one asking for the capital of Japan. A detailed derivation may be useful when the prompt asks for a proof and unsuitable when it asks for a one-sentence answer. There is no single prompt-independent ordering of all possible responses that captures these cases.

This model has the same random-utility interpretation as in Module 1. For a fixed prompt, assign candidate y the realised utility

$$U(x, y) = r_\phi(x, y) + \varepsilon_y.$$

Independent Gumbel noise with a common location and unit scale yields Equation (2.2). With common scale $s > 0$, the logistic argument would instead be the reward difference divided by s ; we fix $s = 1$ to set the units of reward. The reward model represents the systematic part of the comparison, while the random term allows choices to vary across evaluators and occasions.

2.3 Constructing a neural reward model

The contextual Bradley–Terry model tells us what a score should do, but we still need a way to compute that score from text. A reward model can reuse the representations already learned by a language model while changing its output. Let $x \parallel y$ denote the prompt and completed response joined into one token sequence. A pretrained or instruction-tuned Transformer processes this sequence and produces hidden representations. If $h_\psi(x, y) \in \mathbb{R}^d$ is the representation taken at the final non-padding token, a scalar head gives

$$r_\phi(x, y) = w^\top h_\psi(x, y) + b, \quad \phi = (\psi, w, b). \quad (2.3)$$

The final position can incorporate information from the preceding prompt and response through causal attention. Padding must be handled carefully so that the representation is taken from the last actual token rather than the end of the padded array.

The language model and reward model now have distinct jobs. The language model uses a vocabulary-sized output layer repeatedly to generate tokens. The reward model reads the completed prompt–response sequence and returns one scalar. In the response-level model considered here, the comparison outcome supervises this final score; it does not provide a separate reward label for every response token.

One may train the scalar head while keeping the Transformer fixed, or update some or all of the Transformer parameters as well. With a fixed representation, training is a linear comparison problem in w . Updating ψ also allows the representation to adapt to the preference data. Initialising from a model that already represents language avoids asking a much smaller comparison dataset to relearn grammar, meaning, and instruction structure from the beginning. This scalar-head construction is the one used in early systems for reinforcement learning from human feedback (RLHF), with implementation choices differing across systems (Stiennon et al. 2020; Ouyang et al. 2022).

2.3.1 Why merely adding the prompt is not enough

For the score to be contextual, the representation must capture how the response relates to the prompt. This point is easiest to see with separate feature vectors $u = f(x)$ and $v = g(y)$. Suppose the score is additive:

$$r(x, y) = a^\top u + b^\top v + c.$$

When two responses are compared for the same prompt,

$$\begin{aligned} r(x, y^+) - r(x, y^-) &= (a^\top u + b^\top v^+ + c) - (a^\top u + b^\top v^- + c) \\ &= b^\top (v^+ - v^-). \end{aligned}$$

Here $v^+ = g(y^+)$ and $v^- = g(y^-)$. The prompt term cancels. Such a model can learn that one response looks better in general, but its ordering cannot depend on which prompt was asked.

The model needs features that depend jointly on x and y . A Transformer applied to the concatenated sequence can build such features through attention and nonlinear layers. In a simpler feature model with equal-dimensional vectors, one might include the coordinate-wise product $u \odot v$, whose j -th coordinate is $u_j v_j$. Its contribution can differ between (x, y^+) and (x, y^-) , allowing the score to represent prompt–response relevance. The controlled experiment in the course homework isolates this issue.

2.4 From comparison probabilities to a likelihood

With a scoring model in hand, the next step is to fit its parameters to the recorded comparisons. In the contextual Bradley–Terry model, only a reward difference determines a comparison probability. If

$$p = \mathbb{P}_\phi(y_a \succ y_b \mid x),$$

then

$$\log \frac{p}{1-p} = r_\phi(x, y_a) - r_\phi(x, y_b). \quad (2.4)$$

Thus a reward gap is a predicted log-odds. Equal scores give probability $1/2$, and a gap of $\log 3$ gives probability $3/4$. An individual reward is an unrestricted real number; it is not itself a probability.

For observation i , define

$$\Delta_i = r_\phi(x_i, y_i^+) - r_\phi(x_i, y_i^-).$$

Under the model, the probability assigned to the recorded direction is $\sigma(\Delta_i)$. Treating pairwise records as conditionally independent gives the likelihood

$$L_{\text{RM}}(\phi) = \prod_{i=1}^n \sigma(\Delta_i).$$

Maximum likelihood chooses parameters under which the recorded choices are probable. Equivalently, it minimises the average negative log likelihood

$$\mathcal{L}_{\text{RM}}(\phi) = -\frac{1}{n} \sum_{i=1}^n \log \sigma(\Delta_i) = \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-\Delta_i}). \quad (2.5)$$

No numerical rewards appear in $\mathcal{D}_{\text{pref}}$. They are latent scores introduced by the model. Maximum likelihood fits them indirectly, through the comparison probabilities generated by their differences. This is the same estimation principle used for Bradley–Terry utilities in Module 1; the finite utility vector has been replaced by a shared function $r_\phi(x, y)$.

2.5 What the loss learns

The loss specifies the objective; its derivative shows what one comparison asks the model to change. Write $r^+ = r_\phi(x, y^+)$, $r^- = r_\phi(x, y^-)$, and $\Delta = r^+ - r^-$. For the single-comparison loss $\ell = -\log \sigma(\Delta)$, the derivatives are

$$\frac{\partial \ell}{\partial r^+} = \sigma(\Delta) - 1, \quad \frac{\partial \ell}{\partial r^-} = 1 - \sigma(\Delta). \quad (2.6)$$

Gradient descent therefore raises the preferred response's score relative to the other response. If the ordering is confidently wrong, Δ is a large negative number and the derivatives have large magnitude. If the preferred response already has a much larger score, the derivatives are close to zero.

Applying the chain rule gives

$$\nabla_\phi \ell = (\sigma(\Delta) - 1) [\nabla_\phi r_\phi(x, y^+) - \nabla_\phi r_\phi(x, y^-)]. \quad (2.7)$$

The two responses are scored by the same model. Consequently, an update from one pair can alter the scores of other prompt–response pairs which share similar representations.

If the representation is fixed and $r_{w,b}(x, y) = w^\top h(x, y) + b$, then

$$\Delta = w^\top (h(x, y^+) - h(x, y^-)). \quad (2.8)$$

The bias cancels from every reward difference and therefore receives no learning signal from the comparison loss. Fitting w is logistic regression on differences of prompt–response representations. This is the model implemented in the first part of the course homework.

The fixed-feature loss is convex in w , but, as in Module 1, a finite minimiser need not exist. If some w gives every recorded pair a strictly positive gap, scaling it up drives all the loss terms towards zero without attaining zero at finite scores. Parameter penalties or early stopping control this growth in practice. If the representation is trained as well, convexity in the full parameter vector is generally lost.

2.5.1 Which parts of the reward are identified?

A fitted reward is not unique in every respect. The comparisons constrain differences between responses to the same prompt, but they do not determine a separate absolute origin for each prompt. To see this, let $c(x)$ be any function for which the model class can represent both r and the shifted score

$$\tilde{r}(x, y) = r(x, y) + c(x).$$

Then

$$\tilde{r}(x, y_a) - \tilde{r}(x, y_b) = r(x, y_a) - r(x, y_b).$$

The two scores give exactly the same comparison probabilities. At the level of the observations, a common prompt-dependent offset is therefore unidentified. An absolute zero of reward must be chosen separately if a later calculation needs one.

Multiplying reward differences by a positive constant leaves their ordering unchanged but changes the predicted probabilities. A larger positive gap expresses greater confidence in the same preference direction. Reward scale also affects the trade-off between reward and regularisation in Chapter 3.

Exercise 2.E1 — Offsets and scale

For scores $r^+ = 2$ and $r^- = 1$, compute the predicted preference odds. Repeat after adding 10 to both scores, and after multiplying both scores by 2. Which transformation preserves the comparison probability?

2.6 How comparison data are constructed

Before treating $\mathcal{D}_{\text{pref}}$ as a table of triples, it is worth asking how those triples were produced. Prompts determine which tasks are represented. A generating model supplies candidate responses. Labelers apply a rubric through an annotation interface. The learned score is tied to all of these choices.

Candidates may be compared in pairs or ranked in groups. A strict ranking

$$y_1 \succ y_2 \succ \cdots \succ y_K$$

contains $\binom{K}{2}$ pairwise relations. One common loss averages their Bradley–Terry terms:

$$\mathcal{L}_{\text{rank}}(\phi) = -\frac{1}{\binom{K}{2}} \sum_{a < b} \log \sigma(r_\phi(x, y_a) - r_\phi(x, y_b)). \quad (2.9)$$

The terms derived from one ranking share the same prompt, responses, and annotation event. Six pairs extracted from a ranking of four responses are therefore not six independent human judgements. Their grouping should be retained when batches, weights, uncertainty estimates, and dataset splits are constructed. InstructGPT used rankings of several responses and processed the induced comparisons prompt by prompt (Ouyang et al. 2022).

Turning a ranking into pairs is a convenient way to reuse the pairwise Bradley–Terry loss. The product of the pairwise probabilities is a *composite likelihood*: it combines probabilities of related observations without being the joint probability of the ranking. A Plackett–Luce ranking model would instead assign a probability to the ordered list as a whole. We use the pairwise reduction because it is simple and common in language-model training, while remembering that the resulting terms are dependent. A labeler who once ranked $A \succ B \succ C \succ D$ might not reproduce all six implied choices if the pairs were presented separately, in a different order, and at different times.

A response marked y^+ is preferred only relative to the displayed y^- . Whether a comparison is useful therefore depends on what the two candidates allow the evaluator to distinguish. Suppose a prompt asks for a short explanation for a beginner. If both responses are correct but one defines an unfamiliar term and the other does not, their comparison gives a fairly focused signal about accessibility. Two poor responses can also give useful information: a labeler may prefer an explicit admission of uncertainty to a fabricated citation. By contrast, if one response is correct, concise, and well written while the other is wrong, long, and confusing, the preference is easy to label but does not reveal which of the several differences mattered. Near-duplicate answers between which a labeler is forced to choose may contribute mostly noise.

The candidate-generating model determines which such contrasts are available. Comparisons are called *on-policy* when the candidates are sampled from the current policy that is to be improved. They concentrate annotation on mistakes and trade-offs that this model actually produces. Candidates from a recent checkpoint may have similar benefits, although they are no longer exactly on-policy once the policy changes. Comparisons from a much older or unrelated model may remain informative, but they can spend annotation effort on behaviours that the current model has already stopped producing.

Dataset splitting must preserve this structure. Suppose four responses to one prompt are ranked and converted into six pairwise rows. If five rows are placed in the training set and the sixth in the test set, the model has already trained on the same prompt and on the responses appearing in the test pair. A high test score would then say little about performance on a new prompt. To evaluate transfer to new prompts, first divide the prompts into training and test sets; then place every response, ranking, and derived pair for a prompt in the same split. This is often called a *grouped* or *prompt-level* split.

2.7 Preference data in practice

The notation (x, y^+, y^-) makes preference datasets look uniform. In practice, the origin of the preference can differ substantially from one dataset to another. In modern alignment datasets the candidates are often generated by language models; other datasets reuse human-written material from existing platforms. The preference label may come from a direct human comparison, from naturally occurring human behaviour, or from another language model acting as a judge. These sources should not be treated as interchangeable merely because they can all be stored in **chosen** and **rejected** columns.

Direct human comparisons. Several influential public datasets were collected by showing people competing model responses. [HH-RLHF](#) contains pairs selected for helpfulness or harmlessness (Bai et al. 2022). [WebGPT Comparisons](#) records human preferences between long-form answers produced with web browsing (Nakano et al. 2021). [Arena Human Preference 55K](#) contains user votes between two anonymous chatbot responses, including ties (Chiang et al. 2024). In each case, a person saw the alternatives and supplied the comparison outcome. Tied outcomes must be omitted or modelled separately before applying the strict-choice loss derived here.

Human signals inferred from behaviour. Human involvement does not always take the form of a controlled comparison. The [Stanford Human Preferences](#) dataset pairs Reddit answers and infers a collective preference from their scores, with timing restrictions intended to reduce the advantage of earlier comments (Ethayarajh et al. 2022). Its signal is human, but no annotator was presented with the resulting pair and asked to choose between its members. This distinction matters when interpreting what the label represents.

AI feedback and preference distillation. An alternative is to ask a strong language model to evaluate outputs from other models. In [UltraFeedback](#), GPT-4 assigns aspect-level scores and critiques to four responses per prompt (Cui et al. 2023). The [binarised version](#) chooses the response with the highest recorded overall score as y^+ and randomly selects one of the remaining responses as y^- (Tunstall et al. 2023). [Nectar](#) similarly uses GPT-4 to rank seven responses and then derives pairwise comparisons from the ranking (Zhu et al. 2023).

These datasets illustrate a practical form of teacher–student learning. A stronger, and often larger, model acts as the teacher by producing responses, scores, critiques, or rankings. A student model is trained on the resulting demonstrations or preferences, using a reward-model loss or a direct objective such as DPO. The arrangement can make supervision much cheaper to produce, but it also transfers the teacher’s rubric, biases, and blind spots. The mathematical loss is the same whether y^+ was chosen by a person or inferred from an AI score; the provenance of that label must therefore be checked separately.

2.8 What does the learned score represent?

The phrase *learned score* can sound more definite than it is. The model has not discovered an intrinsic quantity called “response quality.” It has learned a numerical summary that helps predict the comparisons in its training data. To understand the score, we must therefore return to the evaluators, instructions, and candidate responses that produced those comparisons.

Choosing between two responses usually requires several judgements at once. Consider a prompt that requests a two-sentence explanation for a beginner. One response may be factually correct but use undefined technical language; another may be clear and follow the requested length but omit an important qualification. There is no single comparison to make until the evaluator decides how to trade accuracy, completeness, accessibility, and instruction following against one another. A different prompt may make safety, citation quality, tone, or formatting relevant instead.

Some datasets make these dimensions explicit. [UltraFeedback](#), for example, records separate assessments of instruction following, truthfulness, honesty, and helpfulness before constructing an overall score (Cui et al. 2023). A simple pairwise dataset records only the final choice, so the trade-off remains implicit. In either case, reducing the judgement to one number discards some information: two responses with the same reward can arrive there for quite different reasons.

The data-collection instructions partly determine how these trade-offs are made. Two evaluators following the same rubric may still place different weight on its criteria. Unless labeler identity is modelled explicitly, the reward model combines their observations. It also sees only the prompts and candidate responses that the collection procedure presented.

For this reason, the learned score should be interpreted as predicting preferences represented by the data-collection procedure. Two groups may prefer different responses; after pooling their labels, the fitted

probability may be close to one half. That number can reflect disagreement in the population rather than indecision by every individual. A further limitation arises if, for one prompt, A beats B , B beats C , and C beats A , each with probability greater than one half. No scalar Bradley–Terry score can fit all three probabilities: it would require $r_A > r_B > r_C > r_A$. An observed cycle in a finite sample is still possible under the model; the restriction concerns the underlying probabilities.

The score therefore summarises choices by the sampled evaluators, under the supplied rubric and interface, over responses from the sampled models and prompts. Changing any of these ingredients can change what it represents.

This interpretation connects directly to the assumptions in Module 1. The contextual Bradley–Terry model provides a useful statistical summary of recorded choices. The meaning of that summary depends on how the observations were elicited and on whether one scalar score is adequate for the intended use.

2.9 Evaluating generalisation and shortcuts

Once a reward model has been fitted, we first ask whether it orders held-out pairs correctly. Pairwise accuracy records how often $r_\phi(x, y^+) > r_\phi(x, y^-)$. Negative log likelihood also measures confidence, so equally accurate models can still have quite different likelihoods.

Held-out pairs test generalisation under one data-collection process. Deployment may instead introduce new prompts, responses from a different model, or labelers using another rubric. These changes should be tested separately where possible.

A flexible reward model may also exploit a shortcut. Length, polished formatting, self-praise, or repetition may correlate with preferred labels without representing the intended judgement. If a stylistic wrapper appears on every preferred response, a response-only model can learn the wrapper rather than relevance. Counterbalancing the style—using it equally often on preferred and dispreferred responses—breaks that correlation; prompt–response interaction features are still needed to learn which answer fits the prompt.

Perturbation tests can keep the prompt fixed while changing formatting, repetition, or a factual claim, or keep the response fixed while replacing its prompt. The reward changes may reveal failures hidden by average accuracy.

Selection tests extrapolation more directly. If the original response remains among several edits, the highest predicted reward cannot fall below its score. That guaranteed increase does not show that the selected text is better; fresh comparisons or task-specific measurements are needed. Repeated policy training magnifies the problem by moving toward poorly covered regions with high predicted reward. Chapter 3 therefore limits departure from a reference policy.

2.10 Summary and key equations

A preference dataset contains comparison outcomes rather than numerical rewards. A reward model turns these observations into a shared scoring function so that preference information can generalise to new prompt–response pairs. The contextual Bradley–Terry model applies the comparison theory from Module 1 with $r_\phi(x, y)$ in place of a fixed item utility. A neural implementation processes the prompt and response jointly and maps their representation to one scalar.

Maximum likelihood fits reward differences to the observed preference directions. Pairwise data leave prompt-dependent offsets unidentified. Preference labels may come from direct human comparisons, behavioural traces, or scores and rankings supplied by an AI judge; the same training loss can be applied to all three. The resulting scalar represents the label source, rubric, prompts, and candidate responses in the collection procedure. Held-out likelihood and accuracy are useful, but prompt transfer, response perturbations, shortcut tests, and evaluation after selection reveal failures that aggregate pairwise metrics may miss.

Key equations

$$\begin{aligned}\mathbb{P}_\phi(y_a \succ y_b \mid x) &= \sigma(r_\phi(x, y_a) - r_\phi(x, y_b)), \\ \Delta_i &= r_\phi(x_i, y_i^+) - r_\phi(x_i, y_i^-), \\ \mathcal{L}_{\text{RM}}(\phi) &= -\frac{1}{n} \sum_i \log \sigma(\Delta_i), \\ \nabla_\phi \ell_i &= (\sigma(\Delta_i) - 1) \nabla_\phi \Delta_i, \\ [r(x, y_a) + c(x)] - [r(x, y_b) + c(x)] &= r(x, y_a) - r(x, y_b).\end{aligned}$$

Improving a policy with a learned reward

Chapter 2 fitted a reward model $r_\phi(x, y)$, which scores a completed response y to a prompt x . We now use this score to change the language-model policy. The reward model evaluates a generated response; the policy determines the probability that it will be generated.

Policy learning repeats four operations: sample a prompt, generate a response, score it, and update the policy. This chapter first derives the update, then explains why unrestricted optimisation of a learned score is unreliable and introduces a fixed reference policy. The reward model is frozen throughout. We write its score as $r(x, y)$, suppressing ϕ , and optimise only the policy parameters θ .

3.1 From response scores to policy value

For a prompt x , the policy generates a complete response y , including its terminal token, and the reward model returns $r(x, y)$. The same process has two useful descriptions. At the token level, the prompt and current prefix form the state and the next token is the action. We set intermediate rewards to zero and receive $r(x, y)$ when generation ends. With no discounting of later rewards, the total reward of the token sequence is the response score. At the response level used below, the sequence of token choices is represented by the distribution $\pi_\theta(y \mid x)$ over completed responses.

The *value* of the policy at prompt x is its expected response reward:

$$V_\theta(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)}[r(x, y)] = \sum_y \pi_\theta(y \mid x) r(x, y). \quad (3.1)$$

The sum ranges over all allowed completed responses. Value is therefore a property of the policy's distribution, not of whichever response happened to appear on one run.

For example, suppose a toy policy can produce only responses A and B , with rewards $r(x, A) = 2$ and $r(x, B) = 0$. If it assigns probability 0.25 to A , its value is 0.5. Raising that probability to 0.75 raises the value to 1.5, even though the reward function has not changed. The second policy can still generate B ; it simply does so less often.

Let μ denote the distribution of prompts on which we want to improve the policy. It could describe a population of future requests or, during training, the empirical distribution that samples uniformly from a fixed collection of prompts. The unregularised objective is

$$J_0(\theta) = \mathbb{E}_{x \sim \mu}[V_\theta(x)] = \mathbb{E}_{x \sim \mu, y \sim \pi_\theta(\cdot \mid x)}[r(x, y)]. \quad (3.2)$$

There are two sources of randomness in this expression: which prompt is drawn and which response the policy generates for it. The prompt distribution determines how much different kinds of request matter to the objective. It is distinct from the comparison dataset used to fit r : the comparison data teach the scorer, while μ specifies where the policy should improve. We treat μ as fixed; the policy controls its responses, not which prompts it receives.

3.2 What pure reward maximisation asks for

Before considering neural-network parameters, fix one prompt and suppose there are finitely many possible responses. Write $r(y)$ as shorthand for $r(x, y)$. If we could choose an arbitrary distribution p over the responses, the problem would be

$$\max_p \sum_y p(y) r(y), \quad p(y) \geq 0, \quad \sum_y p(y) = 1.$$

No average can exceed its largest entry:

$$\sum_y p(y)r(y) \leq \sum_y p(y) \max_z r(z) = \max_z r(z).$$

The bound is attained by assigning all probability to a highest-reward response. If the maximiser is unique, the unrestricted optimum is deterministic; with ties, any distribution supported on the maximisers is optimal. Notice that the starting policy's probabilities do not appear in this objective.

This is a calculation about the objective, not a proposed algorithm. A neural policy may be unable to represent the unrestricted optimum, and a training procedure may fail to find it. Concentration would also be reasonable if r captured everything we cared about: repeatedly choosing a genuinely best response would be sensible. The difficulty is that r is an estimate learned from a finite collection of comparisons. We will return to that limitation after developing a way to improve the policy from samples.

3.3 Why a policy gradient is needed

The sum in Equation (3.1) ranges over far too many responses to evaluate exactly. We can nevertheless estimate the objective by sampling. Draw B prompts $x_i \sim \mu$, generate $y_i \sim \pi_\theta(\cdot | x_i)$, and average the rewards. Here B is the batch size, and

$$\frac{1}{B} \sum_{i=1}^B r(x_i, y_i)$$

is a Monte Carlo estimate of $J_0(\theta)$: it approximates an expectation by an average of random samples. No new human comparison is required for each response at this stage; the frozen reward model supplies its score. Estimating the value of the current policy, however, is not yet a way to improve it.

For a fixed sampled text y , the reward $r(x, y)$ has no gradient with respect to the generator parameters. The reward model is frozen, and a sampled token is a discrete choice rather than a continuously varying input through which ordinary backpropagation can pass. The relevant dependence on θ lies in the probability that the policy assigned to generating y .

Assume for the derivation that the relevant sums are finite and that the policy assigns positive probability to every response under consideration. Because the reward model is fixed, $r(x, y)$ is constant when differentiating with respect to θ . The log-derivative identity

$$\nabla_\theta \pi_\theta(y | x) = \pi_\theta(y | x) \nabla_\theta \log \pi_\theta(y | x)$$

turns the gradient of a sum into an expectation:

$$\begin{aligned} \nabla_\theta V_\theta(x) &= \sum_y r(x, y) \nabla_\theta \pi_\theta(y | x) \\ &= \sum_y \pi_\theta(y | x) r(x, y) \nabla_\theta \log \pi_\theta(y | x) \\ &= \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [r(x, y) \nabla_\theta \log \pi_\theta(y | x)]. \end{aligned} \tag{3.3}$$

This is the *policy-gradient identity*, also called the score-function identity. The original derivative appears to require a sum over all responses. The final expression is an expectation under the current policy, so it can be estimated using responses that the policy generates (Williams 1992).

At iteration k , draw prompts $x_i \sim \mu$ and responses $y_i \sim \pi_{\theta_k}(\cdot | x_i)$. The responses are sampled from the same policy at which the gradient will be evaluated. Such samples are called *on-policy*. Here sampling uses the model distribution itself: changing the temperature, truncating the token distribution, or using greedy decoding would change the sampling distribution and invalidate this estimator unless the calculation were adjusted accordingly. An estimator of the full gradient is

$$\hat{g}_k = \frac{1}{B} \sum_{i=1}^B r(x_i, y_i) \nabla_\theta \log \pi_\theta(y_i | x_i)|_{\theta=\theta_k}. \tag{3.4}$$

Gradient ascent makes the update $\theta_{k+1} = \theta_k + \eta \hat{g}_k$, where $\eta > 0$ is a step size. The log-probability gradient points locally in the direction that would make the sampled response more likely. Multiplying it by a positive weight reinforces that response; multiplying it by a negative weight moves in the opposite direction. For independent samples drawn as specified, the batch average is an unbiased estimate of $\nabla_{\theta} J_0(\theta_k)$. The sign of a raw reward is not intrinsically meaningful, because Chapter 2 showed that its origin can be shifted separately for each prompt. A baseline supplies a centred and usually less noisy weight.

Unbiased does not mean noiseless or guaranteed to improve the objective after every batch. A finite sample can point in an unhelpful direction, and even a useful direction can be followed too far. Both the estimator and the numerical update therefore need more care than the identity alone suggests.

3.3.1 Baselines and response-level advantages

The comparison model in Chapter 2 identified reward differences, not an absolute origin for every prompt. Policy gradients have a matching invariance. Let $b(x)$ be any quantity that depends on the prompt but not on which response was sampled, and treat it as fixed while taking the policy gradient. Then

$$\mathbb{E}_{y \sim \pi_{\theta}} [b(x) \nabla_{\theta} \log \pi_{\theta}(y | x)] = b(x) \nabla_{\theta} \sum_y \pi_{\theta}(y | x) = 0.$$

Consequently,

$$\nabla_{\theta} V_{\theta}(x) = \mathbb{E}_{y \sim \pi_{\theta}} [(r(x, y) - b(x)) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (3.5)$$

Subtracting a *baseline* does not change the expected gradient, but a well-chosen baseline can reduce its variance. If $b(x) = V_{\theta}(x)$, the weight

$$A_{\theta}(x, y) = r(x, y) - V_{\theta}(x)$$

is a response-level *advantage*: it is positive when the sampled response scored above the policy’s average for that prompt and negative when it scored below. In practice the value is unknown and must itself be estimated, for example from other independently sampled responses to the same prompt or by fitting a *value model* that predicts the current policy’s expected reward. This value model is different from the reward model: the reward model scores one completed response, while the value model predicts an average over responses the policy might generate. The estimated baseline is held fixed when differentiating the policy update.

Holding an estimate fixed during differentiation does not by itself make it independent of the sampled response. If a group mean includes the very reward being centred, the resulting estimator is generally biased. For independent responses to one prompt, averaging the other responses’ rewards gives a *leave-one-out baseline*, which preserves the zero expected contribution used above.

This also explains why an unidentified prompt-dependent reward shift is harmless to the ideal objective. Replacing $r(x, y)$ by $r(x, y) + c(x)$ adds the constant $\mathbb{E}_{x \sim \mu}[c(x)]$ to J_0 and leaves its gradient unchanged. In a finite-sample estimator, however, an unnecessary offset can increase noise. Centering rewards with a baseline removes it.

3.4 How a response score reaches token probabilities

The response probability factorises over tokens. Therefore

$$\nabla_{\theta} \log \pi_{\theta}(y | x) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}), \quad (3.6)$$

where the sum includes the terminal token. A response-level advantage consequently weights a sum of ordinary token log-probability gradients. The scalar is shared by all positions in that response, although the gradient at each position is different.

For a sampled pair (x, y) , let $\hat{A}(x, y)$ be the advantage estimate used for that update. A convenient *surrogate loss* is

$$\ell_{\text{PG}}(\theta; x, y) = -\text{sg}[\hat{A}(x, y)] \log \pi_{\theta}(y \mid x), \quad (3.7)$$

where *sg*, for “stop gradient,” means that the sampled advantage is treated as a constant during backpropagation. It is called a surrogate because its gradient supplies the desired policy update; its numerical value is not itself an estimate of negative expected reward. The sampled token IDs are also fixed. Gradients flow through their policy log probabilities, just as they do when likelihood is evaluated, but these tokens were generated by the policy rather than supplied as demonstrations, and their log probabilities are weighted by the estimated advantage.

This basic estimator gives the same response-level weight to every token in a completion. It says that the sequence as a whole performed better or worse than the baseline; it does not identify which particular token deserved credit. Assigning credit more precisely and reducing estimator variance are central practical problems in reinforcement learning.

A collection of generated responses, together with their rewards and log probabilities, is often called a *rollout batch*. Equation (3.4) is unbiased when samples are generated by the policy at which its gradient is evaluated. After the parameters change, the same batch is no longer on-policy. Proximal policy optimisation (PPO) is one practical method for making several controlled updates from a rollout batch: it compares probabilities under the current policy with those under the policy that generated the batch and uses a clipped surrogate objective to limit the incentive for large probability-ratio changes (Schulman et al. 2017). Its detailed update rule is beyond the scope of these notes.

The batch-generating policy is sometimes called the *rollout policy*. It is a recent, temporary copy used to interpret that batch. The *reference policy* introduced next has a different role: it remains fixed throughout reward optimisation and anchors the entire training process to the model from which it began.

3.5 Why pure reward optimisation needs a reference

However broad the preference dataset, it cannot contain every response that the policy might generate. The reward model must therefore generalise: it assigns scores to new prompt–response pairs by extending patterns learned from the comparisons it saw. Chapter 2 introduced this ability as the reason for fitting a reward model rather than memorising a table of observed comparisons. The same generalisation also introduces uncertainty. A pattern that predicts preferences well within the collected data need not remain a reliable guide when optimisation deliberately searches for responses that receive high scores.

The distribution of returned responses can shift even without changing the underlying generator. In *best-of- n selection*, we independently sample n candidate responses from one policy, score all of them, and return the candidate with the largest predicted reward. Increasing n gives the reward model more candidates to search over, so the expected maximum predicted reward cannot decrease. The chance of selecting a response that exploits a scoring error can nevertheless increase. This is a form of reranking.

Policy optimisation can amplify the same problem over many updates. Suppose the comparison data contain a mild accidental association between answer length and human preference. The reward model may then give an unnecessary sentence a small positive bonus. Early policy updates make such sentences a little more common. Those newly generated responses are scored by the same reward model, so later updates can strengthen the tendency. Eventually, the policy may produce far longer answers than the responses on which the apparent length bonus was learned and tested.

The predicted reward can keep rising throughout this process even when people judge the later answers to be no better than the earlier ones, or judge them to be worse. This divergence is called *reward-model over-optimisation*. It is detected by comparing reward-model scores with an independent measure of the quality we actually care about, such as fresh human comparisons or task-specific performance. Gao et al. (2023) studied the phenomenon for both policy optimisation and best-of- n selection in a controlled experiment, using a stronger reward model as a stand-in for human evaluation.

This problem is distinct from overfitting the comparison training set. A scorer can predict held-out

comparisons well and still fail on the different responses found through optimisation. What fails is the link between its score and the intended quality under the changed response distribution.

The starting policy also contains useful behaviour learned during pretraining and supervised fine-tuning. A finite preference dataset cannot enumerate every linguistic ability, factual association, instruction pattern, or appropriate refusal that should be retained. It is therefore useful to make departure from the starting behaviour costly rather than allowing the learned reward to replace it without qualification.

Let $q(y \mid x)$ be a frozen copy of the policy before reward optimisation, usually the SFT policy, and initialise the trainable policy from the same model. We call q the *reference policy*. Its role is to provide an anchor, not another preference judgement. It remains fixed even as π_θ changes. Proximity is measured between the two distributions over responses to the same prompt, rather than by Euclidean distance between their neural-network parameters.

3.6 Kullback–Leibler divergence for response distributions

For a fixed prompt, $\pi(\cdot \mid x)$ and $q(\cdot \mid x)$ are two distributions on the same discrete set of completed responses. Their *Kullback–Leibler (KL) divergence* is

$$\text{KL}(\pi \parallel q) = \sum_y \pi(y \mid x) \log \frac{\pi(y \mid x)}{q(y \mid x)} = \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y \mid x)}{q(y \mid x)} \right]. \quad (3.8)$$

We use natural logarithms. The log ratio compares how much probability the new policy and reference assign to the same response. The expectation is under the new policy, so responses it generates frequently receive more weight in the average.

A two-response example gives some intuition. Suppose $q(A) = 0.8$ and $q(B) = 0.2$. Let δ_A denote the point mass that assigns probability one to A . Moving all probability to A gives

$$\text{KL}(\delta_A \parallel q) = \log(1/0.8) \approx 0.223,$$

whereas moving all probability to B gives $\log(1/0.2) \approx 1.609$. Both new policies are deterministic, but concentrating on the response that the reference considered unusual incurs a larger cost.

The *support* of a distribution is the set of responses to which it assigns positive probability. Terms with $\pi(y \mid x) = 0$ contribute zero to the KL sum. If π assigns positive probability outside the support of q , the divergence is infinite. Otherwise, $\log u \leq u - 1$ gives, with the fixed prompt suppressed,

$$-\text{KL}(\pi \parallel q) = \sum_{y: \pi(y) > 0} \pi(y) \log \frac{q(y)}{\pi(y)} \leq \sum_{y: \pi(y) > 0} (q(y) - \pi(y)) \leq 0.$$

Thus KL divergence is nonnegative. Equality requires $q(y) = \pi(y)$ on the support of π , and no remaining mass outside it, so the divergence is zero exactly when the distributions agree. When both policies use softmax over the same vocabulary and the same length and stopping conventions, every allowed finite response has positive probability under both.

The order of the arguments matters: $\text{KL}(\pi \parallel q)$ is generally not $\text{KL}(q \parallel \pi)$. Here the chosen direction weights the log-probability differences by how often the trainable policy produces each response. An individual sampled log ratio can be negative; it is not by itself a KL divergence. Nonnegativity applies to its expectation, and even a finite-sample average can be negative.

For an autoregressive policy, the response log ratio decomposes as

$$\log \frac{\pi_\theta(y \mid x)}{q(y \mid x)} = \sum_{t=1}^T \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{q(y_t \mid x, y_{<t})}. \quad (3.9)$$

Both models evaluate the same sampled tokens and prefixes. The response log ratio, and hence a sample estimate of the KL, can therefore be computed from token log probabilities. A longer response contains more terms in this sum, but this is not a fixed penalty on length: the terms can have either sign.

3.7 The KL-regularised objective

For $\beta > 0$, consider

$$J_\beta(\theta) = \mathbb{E}_{x \sim \mu} \left[\mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [r(x, y)] - \beta \text{KL}(\pi_\theta(\cdot | x) \| q(\cdot | x)) \right]. \quad (3.10)$$

The first term rewards responses that the learned scorer prefers. The second makes changes in the response distribution costly. During policy optimisation, r , q , μ , and β are fixed; only π_θ changes. A larger β gives more weight to retaining the reference behaviour, while setting $\beta = 0$ recovers pure reward maximisation.

The scale of β is meaningful only relative to the reward scale. Multiplying both the reward and β by the same positive constant multiplies the objective by that constant and leaves its maximisers unchanged. Equivalently, multiplying every reward by $a > 0$ while keeping β fixed has the same maximisers as retaining the original rewards and replacing β by β/a .

The reference itself has zero KL cost. Thus a candidate policy π improves on it under the regularised objective only when its expected reward gain exceeds the KL price of the departure:

$$\mathbb{E}_{x \sim \mu} \left[\mathbb{E}_{y \sim \pi(\cdot | x)} r(x, y) - \mathbb{E}_{y \sim q(\cdot | x)} r(x, y) \right] > \beta \mathbb{E}_{x \sim \mu} \text{KL}(\pi(\cdot | x) \| q(\cdot | x)).$$

The penalty therefore changes which final policy is preferred. Taking small gradient steps is not an equivalent safeguard: many individually small steps can eventually move a policy far from where it began.

KL regularisation is also not a guarantee of correctness, safety, or human approval. It preserves proximity to a chosen reference while the policy pursues a learned score. If the reference has an undesirable behaviour, staying near it can preserve that behaviour as well. The achieved KL is a trade-off determined by the rewards, β , and the policy class, not a prescribed hard upper bound.

3.7.1 The gradient of the regularised objective

At one prompt, write the objective as the expectation of the adjusted response score

$$R_\theta(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y | x)}{q(y | x)}.$$

Unlike the learned reward, this adjusted score depends on θ . Since r and q are fixed,

$$\nabla_\theta R_\theta(x, y) = -\beta \nabla_\theta \log \pi_\theta(y | x).$$

Differentiating its expectation therefore produces both a policy-gradient term, because the sampling distribution depends on θ , and the direct derivative shown above:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{y \sim \pi_\theta} [R_\theta(x, y)] &= \mathbb{E}_{y \sim \pi_\theta} [R_\theta(x, y) \nabla_\theta \log \pi_\theta(y | x) + \nabla_\theta R_\theta(x, y)] \\ &= \mathbb{E}_{y \sim \pi_\theta} [(R_\theta(x, y) - \beta) \nabla_\theta \log \pi_\theta(y | x)]. \end{aligned}$$

The extra $-\beta$ is a constant baseline. Its expected contribution is zero because

$$\mathbb{E}_{y \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(y | x)] = \nabla_\theta \sum_y \pi_\theta(y | x) = 0.$$

We may therefore write

$$\nabla_\theta J_\beta(\theta) = \mathbb{E}_{x \sim \mu, y \sim \pi_\theta} \left[\left(r(x, y) - \beta \log \frac{\pi_\theta(y | x)}{q(y | x)} \right) \nabla_\theta \log \pi_\theta(y | x) \right]. \quad (3.11)$$

For a sampled response, define its regularised, baseline-centred weight as

$$\hat{A}_\beta(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y | x)}{q(y | x)} - b(x).$$

The gradient can then be estimated with the surrogate loss

$$\ell_{\text{reg}}(\theta; x, y) = -\text{sg}\left[\hat{A}_\beta(x, y)\right] \log \pi_\theta(y | x).$$

For this estimator, responses are sampled from the current policy and the log ratio is evaluated at that same policy. The entire weight is then held fixed within the backward pass. This stop-gradient operation makes the surrogate reproduce Equation (3.11); differentiating through the weight would give a different update. Gradients flow through the policy log probability outside the stop-gradient operator. The reward model and reference parameters remain fixed, and the baseline $b(x)$ can be chosen to reduce variance. Recomputing weights on an old rollout batch alone does not restore the on-policy condition.

This chapter has separated the objective from the algorithm used to optimise it. A practical method such as PPO addresses how to estimate advantages and take stable updates from rollout batches. A different mathematical question is what response distribution would maximise J_β if distributions could be chosen freely. Solving that problem reveals the connection between rewards, reference probabilities, and direct preference optimisation.

3.8 Summary and key equations

Expected reward evaluates the complete distribution of responses produced by a policy. The policy-gradient identity converts its derivative into an expectation over sampled responses and differentiable log probabilities. A baseline independent of the sampled response leaves the expected gradient unchanged and can reduce noise. The response probability factorises into the token probabilities computed by the language model.

Pure optimisation of a learned reward can concentrate the policy on errors in that reward. A fixed reference policy imposes a cost for departing from the SFT behaviour, and KL divergence measures that departure at the level of response probabilities. The regularised objective balances higher learned reward against this cost; it does not make the scorer or reference infallible.

Key equations

$$\begin{aligned} V_\theta(x) &= \mathbb{E}_{y \sim \pi_\theta(\cdot | x)}[r(x, y)], \\ \nabla_\theta V_\theta(x) &= \mathbb{E}_{y \sim \pi_\theta(\cdot | x)}[(r(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y | x)], \\ \nabla_\theta \log \pi_\theta(y | x) &= \sum_t \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}), \\ \text{KL}(\pi(\cdot | x) \| q(\cdot | x)) &= \mathbb{E}_{y \sim \pi(\cdot | x)} \left[\log \frac{\pi(y | x)}{q(y | x)} \right], \\ J_\beta(\theta) &= \mathbb{E}_{x \sim \mu} \left[\mathbb{E}_{y \sim \pi_\theta(\cdot | x)} r(x, y) - \beta \text{KL}(\pi_\theta(\cdot | x) \| q(\cdot | x)) \right], \\ \nabla_\theta J_\beta(\theta) &= \mathbb{E}_{x, y} \left[\left(r(x, y) - \beta \log \frac{\pi_\theta(y | x)}{q(y | x)} \right) \nabla_\theta \log \pi_\theta(y | x) \right]. \end{aligned}$$

In the final expectation, $x \sim \mu$ and $y \sim \pi_\theta(\cdot | x)$.

From the optimal policy to direct preference optimisation

Chapter 3 considered a fixed reward model and a trainable policy. One way to improve the policy is to generate responses, score them, and estimate a policy gradient. This chapter takes a different route. We first ask what response distribution would maximise the KL-regularised reward objective if its probabilities could be chosen directly. The answer gives an exact relation among reward, the reference policy, and the optimal policy.

That relation can be read in reverse. Instead of using a learned reward to determine a policy, we can use a trainable policy and a reference policy to parameterise the scores in a preference model. The Bradley–Terry likelihood then becomes a loss on the language model itself. This is the central idea behind direct preference optimisation (DPO) (Rafailov et al. 2023).

4.1 A warm-up: entropy-regularised probability allocation

Consider K alternatives with scores $a_1, \dots, a_K$. We may choose any probability vector $p = (p_1, \dots, p_K)$ in the simplex

$$\Delta_K = \left\{ p : p_i \geq 0, \sum_{i=1}^K p_i = 1 \right\}.$$

Maximising $\sum_i p_i a_i$ places all probability on the alternatives with the largest score. Even a very small score advantage can therefore lead to complete concentration.

To make concentrated distributions costly, add the entropy

$$H(p) = - \sum_{i=1}^K p_i \log p_i, \quad 0 \log 0 = 0.$$

Entropy is zero for a point mass and largest for the uniform distribution. For $\beta > 0$, consider

$$F(p) = \sum_i p_i a_i + \beta H(p) = \sum_i p_i a_i - \beta \sum_i p_i \log p_i. \quad (4.1)$$

The first term favours high-scoring alternatives. The second rewards spreading probability, with β controlling the trade-off.

For finite scores, the optimum assigns positive probability to every alternative. Giving a zero-probability alternative a small mass ε , taken from one with positive mass, gains an entropy term of order $-\varepsilon \log \varepsilon$; this outweighs changes proportional to ε when ε is sufficiently small. At this interior optimum, we can enforce the constraint $\sum_i p_i = 1$ with a Lagrange multiplier λ . The resulting Lagrangian is

$$\mathcal{G}(p, \lambda) = \sum_i p_i a_i - \beta \sum_i p_i \log p_i - \lambda \left(\sum_i p_i - 1 \right).$$

The multiplier represents a common marginal value of probability mass. If two alternatives had different marginal gains, transferring a little probability from the smaller one to the larger would improve the objective without changing the total mass. Differentiating with respect to p_i therefore gives

$$a_i - \beta(1 + \log p_i) - \lambda = 0.$$

Solving this equation shows that p_i is proportional to $\exp(a_i/\beta)$. Normalisation gives

$$p_i^* = \frac{\exp(a_i/\beta)}{\sum_j \exp(a_j/\beta)}. \quad (4.2)$$

The objective is strictly concave on the simplex, so this stationary point is the unique maximiser. We have obtained the softmax distribution from an optimisation problem rather than assuming it as a modelling choice.

The odds form is especially useful:

$$\frac{p_i^*}{p_j^*} = \exp\left(\frac{a_i - a_j}{\beta}\right).$$

Only score differences matter. For fixed scores, decreasing β makes the distribution more sensitive to those differences and eventually concentrates on the score maximisers. Increasing β weakens their effect and moves the distribution towards uniformity.

4.2 Solving the KL-regularised reward objective

We now return to language models. Fix a prompt x and, for the derivation, a finite set of allowed complete responses. Write

$$p_y = \pi(y \mid x), \quad q_y = q(y \mid x), \quad r_y = r(x, y).$$

Assume that all rewards are finite and that q is a probability distribution with $q_y > 0$ for every allowed response. If the reference assigned zero probability to some response, placing positive policy probability there would give infinite $\text{KL}(p\|q)$; equivalently, the optimisation would have to be restricted to the support of q .

The one-prompt objective from Chapter 3 is

$$\begin{aligned} F_x(p) &= \sum_y p_y r_y - \beta \sum_y p_y \log \frac{p_y}{q_y} \\ &= \sum_y p_y (r_y + \beta \log q_y) + \beta H(p). \end{aligned}$$

This is exactly the entropy-regularised problem just solved, with adjusted score $r_y + \beta \log q_y$. Substitution into (4.2) gives

$$\pi_r^*(y \mid x) = \frac{q(y \mid x) \exp(r(x, y)/\beta)}{Z_r(x)}, \quad Z_r(x) = \sum_{y'} q(y' \mid x) \exp(r(x, y')/\beta). \quad (4.3)$$

The subscript r records that this is the optimum for the particular reward function r . The normalising factor $Z_r(x)$, sometimes called a partition function, makes the probabilities sum to one. A finite vocabulary and a fixed stopping limit give a finite response set, as assumed here.

Equation (4.3) has a simple interpretation. Begin with the reference probability of each response, multiply it by an exponential reward factor, and then renormalise. The reference is not discarded. It supplies the starting weights to which the reward is applied.

For two responses,

$$\frac{\pi_r^*(y_a \mid x)}{\pi_r^*(y_b \mid x)} = \frac{q(y_a \mid x)}{q(y_b \mid x)} \exp\left(\frac{r(x, y_a) - r(x, y_b)}{\beta}\right). \quad (4.4)$$

A reward difference therefore multiplies the reference odds. A response with higher reward need not become the more probable response if the reference initially placed much greater probability on its competitor.

For a finite response set, $\beta \rightarrow \infty$ returns the reference distribution. As $\beta \downarrow 0$, probability concentrates on the reward maximisers; if several responses tie, their limiting relative weights are inherited from the reference. Adding the same constant $c(x)$ to every reward for one prompt multiplies both the numerator and $Z_r(x)$ by the same factor and leaves the optimal policy unchanged.

4.2.1 An exact optimality gap

The closed-form solution also proves its own optimality. Taking logarithms in (4.3) gives

$$\log \frac{\pi_r^*(y | x)}{q(y | x)} = \frac{r(x, y)}{\beta} - \log Z_r(x).$$

Substituting this identity into the objective for any admissible response distribution π gives

$$F_x(\pi) = \beta \log Z_r(x) - \beta \text{KL}(\pi \| \pi_r^*). \quad (4.5)$$

Since KL divergence is nonnegative, the largest possible value is $\beta \log Z_r(x)$, attained at $\pi = \pi_r^*$. More precisely,

$$F_x(\pi_r^*) - F_x(\pi) = \beta \text{KL}(\pi \| \pi_r^*). \quad (4.6)$$

The gap is therefore not merely bounded by a divergence; it is exactly a divergence multiplied by β .

4.2.2 Why this is not yet a training algorithm

The derivation allowed a separate, freely chosen distribution for every prompt. A language model is much more constrained: one parameter vector θ determines the distributions for all prompts, so changing one response probability generally changes many others. In addition, $Z_r(x)$ requires a sum over an enormous set of complete responses. Even if the reward model were known, (4.3) would not tell us which neural-network parameters realise the resulting distribution.

The closed form is nevertheless useful. It identifies an exact relation that any unrestricted optimum must satisfy. DPO uses that relation to construct a likelihood that can be optimised with ordinary backpropagation.

4.3 From an optimal policy to a preference likelihood

Equation (4.3) maps a reward function to its regularised optimal policy. To read the relation in the other direction, take logarithms and rearrange:

$$r(x, y) = \beta \log \frac{\pi_r^*(y | x)}{q(y | x)} + \beta \log Z_r(x). \quad (4.7)$$

The last term depends on the prompt but not on the response. It therefore cancels when two responses to the same prompt are compared:

$$r(x, y^+) - r(x, y^-) = \beta \left[\log \frac{\pi_r^*(y^+ | x)}{q(y^+ | x)} - \log \frac{\pi_r^*(y^- | x)}{q(y^- | x)} \right]. \quad (4.8)$$

This is the same prompt-dependent offset invariance that appeared in Chapter 2. It is also why the derivation compares responses to the same prompt: only then does the unknown normalising term cancel. DPO replaces the unrestricted optimum in this difference by a trainable policy. Define the policy's implicit comparison score

$$s_\theta(x, y) = \beta \log \frac{\pi_\theta(y | x)}{q(y | x)}. \quad (4.9)$$

This score records how much the policy has raised or lowered the log probability of y relative to the reference. It is called an implicit reward because its differences play the role that reward differences played in the Bradley–Terry model. We assume that both policies assign positive probability to every allowed response, so the log ratio is defined.

This parameterisation retains the earlier reward–policy relation. For the reward s_θ , the unnormalised optimal-policy weight is

$$q(y | x) e^{s_\theta(x, y)/\beta} = \pi_\theta(y | x).$$

These weights already sum to one, so π_θ is exactly optimal for its own implicit reward. Conversely, (4.7) shows that every finite reward function on our finite response set has such a representation up to a prompt-dependent offset, if the policy distribution can be chosen freely.

For a recorded comparison $y^+ \succ y^-$, the corresponding margin is

$$m_\theta(x, y^+, y^-) = s_\theta(x, y^+) - s_\theta(x, y^-) = \beta \left[\log \frac{\pi_\theta(y^+ | x)}{q(y^+ | x)} - \log \frac{\pi_\theta(y^- | x)}{q(y^- | x)} \right]. \quad (4.10)$$

The Bradley–Terry model assigns probability $\sigma(m_\theta)$ to the recorded preference. Maximum likelihood on $\mathcal{D}_{\text{pref}} = \{(x_i, y_i^+, y_i^-)\}_{i=1}^n$ therefore gives

$$\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{n} \sum_{i=1}^n \log \sigma(m_\theta(x_i, y_i^+, y_i^-)). \quad (4.11)$$

This is the reward-model likelihood from Chapter 2, with $s_\theta(x, y)$ in place of a separately parameterised reward $r_\phi(x, y)$. The preference triples, reference policy, and β are fixed; the parameters of π_θ are learned. The partition function has vanished because the likelihood uses within-prompt score differences.

One point in this argument deserves care. We began with a fixed reward r and derived its unrestricted optimum π_r^* . When training DPO, we do not know that reward and do not declare each intermediate π_θ to be optimal for it. Instead, $\beta \log(\pi_\theta/q)$ is a parameterisation of the comparison score, and that score changes as θ changes. DPO learns a policy whose implicit score differences explain the recorded comparisons. The derivation motivates this parameterisation; finite data, shared neural parameters, and numerical optimisation still determine what can actually be learned.

4.4 What the DPO margin asks the policy to change

Regrouping the terms in (4.10) gives

$$m_\theta = \beta \log \left(\frac{\pi_\theta(y^+ | x)/\pi_\theta(y^- | x)}{q(y^+ | x)/q(y^- | x)} \right). \quad (4.12)$$

The numerator contains the current policy’s preferred-versus-dispreferred odds; the denominator contains the reference odds for the same pair. Thus DPO asks whether the policy has shifted relative probability towards the recorded preferred response.

If the policy equals the reference, every margin is zero, the predicted comparison probability is 1/2, and the loss for each pair is $\log 2$. This remains true when the reference itself assigns unequal probabilities to the two responses. A positive DPO margin means that the policy favours y^+ more strongly than the reference did. It does not necessarily mean that $\pi_\theta(y^+ | x) > \pi_\theta(y^- | x)$.

Example 4.4.1 (Improved relative odds). Suppose there are two displayed responses, with reference probabilities $q(y^+ | x) = 0.2$ and $q(y^- | x) = 0.8$. A policy assigning probabilities 0.4 and 0.6 changes their odds from 1/4 to 2/3. The preferred response is still generated less often than the dispreferred one, but its relative odds have improved by a factor of 8/3. With $\beta = 1$,

$$m_\theta = \log(8/3), \quad \sigma(m_\theta) = \frac{8}{11} \approx 0.727.$$

The value 0.727 is the preference probability predicted by the Bradley–Terry model. The generation probabilities remain 0.4 and 0.6; the two quantities describe different events.

Zero is not a target at which learning stops. The loss $\ell(m) = -\log \sigma(m)$ continues to favour larger positive margins, although its slope becomes smaller once the recorded preference is already predicted with high probability. Since

$$\frac{d\ell}{dm} = -\sigma(-m),$$

differentiation of one pair gives

$$\nabla_{\theta} \ell_{\text{DPO}} = -\beta \sigma(-m_{\theta}) \left[\nabla_{\theta} \log \pi_{\theta}(y^+ | x) - \nabla_{\theta} \log \pi_{\theta}(y^- | x) \right]. \quad (4.13)$$

The reference has no gradient. The coefficient $\sigma(-m_{\theta})$ is large when the margin is poor and small when the pair is already well ordered. For one pair, a sufficiently small gradient step increases the preferred response’s log probability relative to the dispreferred response’s, unless the gradient is zero. This need not increase the preferred response’s absolute probability: both probabilities could fall while their ratio increases. Shared parameters also couple this update to other pairs, whose gradients may conflict.

The role of β can otherwise be easy to misread. In the distributional problem, a larger β places a larger penalty on departure from the reference. In the DPO margin, the same β scales the policy-to-reference log-ratio difference. For a fixed reward difference, a smaller β corresponds through (4.4) to a larger change in policy odds. During finite-data training, however, β also changes the logistic margin and its gradient. Its practical effect is therefore not identical to changing only the learning rate. The DPO loss contains no separately evaluated KL penalty, and reducing this loss on recorded pairs gives no fixed bound on the policy’s KL divergence from the reference.

4.5 Computing a DPO training step

A minibatch contains recorded triples (x, y^+, y^-) . For each triple, the loss needs four response log probabilities:

$$\log \pi_{\theta}(y^+ | x), \quad \log \pi_{\theta}(y^- | x), \quad \log q(y^+ | x), \quad \log q(y^- | x).$$

The two reference values are fixed and may be computed once and cached when the dataset, tokenisation, and prompt formatting remain unchanged. The two policy values must be recomputed as the trainable model changes.

For either supplied response $y = (y_1, \dots, y_T)$, including its terminal token,

$$\log \pi_{\theta}(y | x) = \sum_{t=1}^T \log \pi_{\theta}(y_t | x, y_{<t}). \quad (4.14)$$

The response is not sampled during this calculation. The recorded prompt and response are fed to the causal model, which predicts the next token at every prefix in one forward pass. This evaluation of supplied tokens is often called teacher forcing.

An implementation must align each output position with the following observed token, apply log-softmax over the vocabulary, select the log probability of that observed token, and sum over response positions. A mask excludes prompt tokens and padding and includes the desired termination token. Averaging rather than summing over response positions would produce a length-normalised score and therefore a different objective. The policy and reference must use the same tokenisation, chat template, response boundary, and terminal-token convention.

Once the four log probabilities are available, compute (4.10), apply the negative log-sigmoid, average across the minibatch, and backpropagate through the policy scores. No gradient is taken through the fixed reference scores. The course homework implements precisely these two ingredients: summed response log probabilities and the pairwise DPO loss.

4.6 What DPO changes, and what it leaves unchanged

Reward-based policy optimisation and DPO use the same underlying language-model probabilities, but they obtain their update signals differently. Reward-based training generates responses from a policy, evaluates them with a separately learned reward model, estimates advantages, and applies a policy-gradient method. The basic DPO procedure studied here evaluates recorded preferred and dispreferred

responses and directly differentiates their preference likelihood. It needs neither a separate reward network nor a value model or rollout batch for each update.

The word *direct* refers to this path from comparisons to policy parameters. It does not mean that DPO is free of modelling assumptions. The Bradley–Terry likelihood, the reference policy, the value of β , and the construction of the preference pairs all remain part of the method. The policy itself supplies an implicit comparison score rather than eliminating the idea of a score.

There is also a data trade-off. A learned reward model can score fresh responses as the policy changes, although Chapter 3 explained why those scores may become unreliable away from its training distribution. Standard offline DPO trains on a fixed set of pairs, so it does not automatically obtain judgements about the new responses produced by the changing policy. New comparisons can be collected and training repeated, but the fixed-dataset objective alone does not perform that refresh.

Both approaches therefore depend on the coverage and provenance of their preference data. If the comparisons reward a superficial style, omit important prompt types, or combine incompatible judgements, changing the optimisation method does not restore the missing information. A lower training loss establishes that the recorded comparisons are better predicted. It does not by itself establish that newly generated answers are more truthful, helpful, or safe. Evaluation must return to fresh prompts, generated responses, and measurements that were not used as the training signal.

Some tasks provide a different form of feedback altogether. Code may be checked by tests, or a mathematical answer by a verifier. Such outcomes can supply rewards for generated attempts without a subjective pairwise comparison. They belong to the broader post-training picture discussed in the lectures, but their reliability and the behaviour induced by optimising them require a separate treatment.

4.7 Summary and key equations

Entropy-regularised probability allocation yields a softmax distribution. Applying the same result to a KL-regularised reward objective gives an exponential reweighting of the reference policy. Inverting this relation expresses reward through a policy-to-reference log ratio, up to a prompt-dependent constant. That constant cancels in comparisons between responses to the same prompt. Substituting the resulting implicit score into the Bradley–Terry likelihood produces DPO.

DPO trains directly on recorded preference pairs with ordinary backpropagation. Its margin measures how much the policy has shifted the preferred-versus-dispreferred odds relative to the reference. It removes the separate reward-model and policy-gradient stages from this training objective, but it does not remove assumptions about the preference data or the need to evaluate the resulting policy on generated responses.

Key equations

$$\pi_r^*(y \mid x) = \frac{q(y \mid x)e^{r(x,y)/\beta}}{Z_r(x)},$$

$$Z_r(x) = \sum_{y'} q(y' \mid x)e^{r(x,y')/\beta},$$

$$s_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{q(y \mid x)},$$

$$m_\theta(x, y^+, y^-) = s_\theta(x, y^+) - s_\theta(x, y^-),$$

$$\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{n} \sum_i \log \sigma(m_{\theta,i}),$$

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \sigma(-m_\theta) \nabla_\theta \left[\log \pi_\theta(y^+ \mid x) - \log \pi_\theta(y^- \mid x) \right].$$

Historical remarks

The ideas in these notes come from several research traditions that developed for different purposes. Statistical language modelling, pairwise comparison models, policy-gradient reinforcement learning, and reward learning from human feedback did not begin as one unified programme. The account below is selective. It identifies developments that help explain the particular mathematical path followed in this module, without assigning a single origin to broad ideas that often appeared in related forms elsewhere.

Statistical generation before neural networks

Shannon (1948) used progressively richer statistical approximations to generate English-like text. One word-level approximation samples words independently according to their frequencies. Another conditions each word on its predecessor. He reported examples such as:

Independent words: “REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN”.

Word transitions: “THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER”.

The transition model produces more plausible local fragments because the next word depends on the current one, although long-range organisation remains weak.

Shannon described sampling procedures that could be carried out with books rather than a stored numerical table. For a word-transition approximation, one finds an occurrence of the current word and records the word that follows it, then repeats from the new word. A modern idealisation makes the empirical transition probability explicit. If $c(u, v)$ counts occurrences of v immediately after u , then

$$p(v \mid u) = \frac{c(u, v)}{\sum_{v'} c(u, v')}.$$

For a word u with at least one recorded successor, selecting uniformly among such occurrences reproduces this conditional distribution. Repeating the operation generates a sequence one word at a time. The vocabulary, scale, and mechanism have changed enormously, but the idea of defining a distribution for the next symbol from its preceding context remains at the centre of language modelling.

Pairwise comparisons as statistical observations

The comparison model used throughout these notes has a separate history. Bradley and Terry (1952) introduced a probabilistic model for paired-comparison outcomes in which differences of latent scores determine win probabilities. Module 1 developed this model, its invariances, and its likelihood in detail.

Bradley and Terry were not studying language-model alignment. The modern connection is mathematical: a preferred and dispreferred response can be treated as two alternatives in a comparison, while a neural network supplies scores that generalise across prompts and responses. Chapter 2 replaced one free utility per item by the contextual score $r_\phi(x, y)$; Chapter 4 replaced that explicit score by a policy-to-reference log ratio. In both cases, the same logistic comparison likelihood turns observed choices into an estimable objective.

Neural language models and sequence-level learning

Bengio et al. (2003) developed a neural probabilistic language model with learned word representations, a softmax next-word distribution, and likelihood-based training. The Transformer architecture of Vaswani et al. (2017) later provided the attention-based backbone used by most of the language models discussed in this module. Large-scale pretraining then made one model useful across many tasks; GPT-3 demonstrated striking zero-shot and few-shot behaviour without changing its parameters for each task (Brown et al. 2020).

Next-token likelihood is not the only possible training criterion for a sequence generator. The policy-gradient identity studied in Chapter 3 belongs to reinforcement learning and predates its use for language models. Williams (1992) developed the REINFORCE family of gradient estimators for stochastic networks. In sequence generation, the sampled tokens can be viewed as actions and a score for the completed sequence as a return. Ranzato et al. (2016), first circulated in 2015, gave an influential explicit treatment of recurrent sequence generation in these terms and used policy-gradient ideas to optimise sequence-level evaluation measures.

Proximal policy optimisation was introduced by Schulman et al. (2017) as a general reinforcement-learning method, with experiments on simulated control and Atari rather than language-model alignment. Its clipped surrogate allowed several minibatch updates from sampled experience. PPO was later adopted in prominent language-model feedback pipelines. This chronology is useful because PPO is an optimisation method used inside some RLHF systems; it was not itself invented as a theory of human preferences or language.

Learning rewards from human comparisons

Christiano et al. (2017) studied agents for which writing the desired reward by hand was difficult. Human evaluators compared short behaviour segments from Atari games and simulated robot locomotion. A reward predictor learned from those comparisons, and reinforcement learning then improved the policy using the predicted reward. The evaluator did not have to score every interaction: the learned model generalised a limited number of comparisons to many subsequent behaviours. The same economy also created the risk that policy optimisation would exploit errors in the predictor.

The transfer of this template to language happened in stages. Ziegler et al. (2019) applied reward learning from human comparisons to pretrained language models on stylistic continuation and summarisation tasks. Their results already showed both the attraction of the method and a recurring difficulty: a model can perform well under the elicited judgements while making use of regularities in how those judgements were supplied. Stiennon et al. (2020) developed the summarisation setting further, collecting comparisons between model summaries, fitting a reward model, and optimising a summarisation policy against it.

Subsequent work broadened the tasks and the role of the model. WebGPT combined language-model generation with web browsing and trained from human comparisons of long-form answers (Nakano et al. 2021). Work on helpful and harmless assistants collected dialogue preferences and studied reward-based fine-tuning for these separate criteria (Bai et al. 2022). InstructGPT provided an especially clear and influential three-stage account: supervised fine-tuning on demonstrations, reward modelling from rankings of model responses, and PPO against the learned reward with a penalty for departure from the supervised model (Ouyang et al. 2022).

These systems differed in prompts, interfaces, evaluator instructions, model sizes, and optimisation details. Their common structure is the one isolated in Chapters 2 and 3: comparisons train a scoring rule, and that rule supplies many more evaluations than people could provide directly. Their history also reinforces the interpretive point of Chapter 2. The learned reward represents preferences expressed through a particular data-collection procedure, not a context-free measurement of everything desirable in an answer.

Direct preference objectives

Reward modelling and policy optimisation introduce two learned objects and a multi-stage training loop. Rafailov et al. (2023) showed that the KL-regularised objective admits another route. The optimal response distribution for a fixed reward is an exponential tilt of the reference policy. Inverting that relation expresses reward differences through policy-to-reference log ratios. Substitution into a Bradley–Terry likelihood yields DPO: the policy can be trained on preference pairs without first fitting a separate reward network or sampling rollouts during each update.

The methods continue to serve different roles. Demonstrations provide examples to imitate. Reward-based methods can evaluate newly generated responses through a learned scorer. Direct methods avoid that separate scorer but rely more immediately on the coverage of recorded pairs. DPO offered a particularly simple objective for pairwise preference data, a fixed reference policy, and an offline likelihood-based update. Modern post-training recipes often combine supervised demonstrations, preference optimisation, synthetic or human feedback, and rewards from automatically checkable outcomes.

Seen together, the historical threads explain the structure of this module. Autoregressive language modelling supplies probabilities for complete responses. Pairwise comparison theory turns choices into likelihoods. Policy gradients allow scores on generated sequences to change a stochastic generator. KL regularisation relates reward improvements to departure from a reference. DPO closes the circle by writing the comparison score itself in terms of language-model probabilities.

References

- Bai, Yuntao et al. (2022). “Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback”. In: *arXiv preprint arXiv:2204.05862*. URL: <https://arxiv.org/abs/2204.05862>.
- Bengio, Yoshua, Rejean Ducharme, Pascal Vincent, and Christian Jauvin (2003). “A Neural Probabilistic Language Model”. In: *Journal of Machine Learning Research* 3, pp. 1137–1155. URL: <https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>.
- Bradley, Ralph Allan and Milton E. Terry (1952). “Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons”. In: *Biometrika* 39.3/4, pp. 324–345. DOI: [10.2307/2334029](https://doi.org/10.2307/2334029).
- Brown, Tom B. et al. (2020). “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2005.14165>.
- Chiang, Wei-Lin et al. (2024). “Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference”. In: *arXiv preprint arXiv:2403.04132*. URL: <https://arxiv.org/abs/2403.04132>.
- Christiano, Paul F. et al. (2017). “Deep Reinforcement Learning from Human Preferences”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/1706.03741>.
- Cui, Ganqu et al. (2023). “UltraFeedback: Boosting Language Models with Scaled AI Feedback”. In: *arXiv preprint arXiv:2310.01377*. URL: <https://arxiv.org/abs/2310.01377>.
- Ethayarajh, Kavin, Yejin Choi, and Swabha Swayamdipta (2022). “Understanding Dataset Difficulty with V-Usable Information”. In: *Proceedings of the 39th International Conference on Machine Learning*, pp. 5988–6008. URL: <https://proceedings.mlr.press/v162/ethayarajh22a.html>.
- Gao, Leo, John Schulman, and Jacob Hilton (2023). “Scaling Laws for Reward Model Overoptimization”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, pp. 10835–10866. URL: <https://proceedings.mlr.press/v202/gao23h.html>.
- Nakano, Reiichiro et al. (2021). “WebGPT: Browser-Assisted Question-Answering with Human Feedback”. In: *arXiv preprint arXiv:2112.09332*. URL: <https://arxiv.org/abs/2112.09332>.
- Ouyang, Long et al. (2022). “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2203.02155>.
- Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2305.18290>.
- Ranzato, Marc’Aurelio, Sumit Chopra, Michael Auli, and Wojciech Zaremba (2016). “Sequence Level Training with Recurrent Neural Networks”. In: *International Conference on Learning Representations*. URL: <https://arxiv.org/abs/1511.06732>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (2017). “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347*. URL: <https://arxiv.org/abs/1707.06347>.
- Shannon, Claude E. (1948). “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27, 379–423 and 623–656. URL: <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>.
- Stiennon, Nisan et al. (2020). “Learning to summarize from human feedback”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2009.01325>.
- Tunstall, Lewis et al. (2023). “Zephyr: Direct Distillation of LM Alignment”. In: *arXiv preprint arXiv:2310.16944*. URL: <https://arxiv.org/abs/2310.16944>.
- Vaswani, Ashish et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/1706.03762>.
- Williams, Ronald J. (1992). “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8, pp. 229–256. DOI: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696). URL: <https://doi.org/10.1007/BF00992696>.
- Zhu, Banghua, Evan Frick, Tianhao Wu, Hanlin Zhu, and Jiantao Jiao (2023). *Nectar Dataset Card*. URL: <https://huggingface.co/datasets/berkeley-nest/Nectar> (visited on 09/22/2026).
- Ziegler, Daniel M., Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving (2019). “Fine-Tuning Language Models from Human Preferences”. In: *arXiv preprint arXiv:1909.08593*. URL: <https://arxiv.org/abs/1909.08593>.